1. Общая часть

1.1. Анализ предметной области и обоснование актуальности разработки

Современные системы хранения и дистрибуции подкастов представляют собой сложные программные комплексы, работающие по модели «клиент-сервер». Основная задача таких систем — предоставить пользователю быстрый доступ к аудиофайлам без необходимости их предварительного скачивания целиком на устройство, что реализуется с помощью технологии прогрессивного скачивания (progressive download) или потокового вещания (streaming).

С точки зрения контент-менеджмента, предметная область характеризуется наличием двух основных ролей:

- слушатель (Client/Listener): регистрируется в системе, осуществляет поиск подкастов по ключевым словам, фильтрует каталог по категориям и прослушивает аудиозаписи с помощью интерактивного плеера;
- администратор (Admin): осуществляет глобальное администрирование системы, включая просмотр пользователей, добавление новых подкастов (с метаданными и аудиофайлами) и удаление устаревшего контента.

Актуальность разработки собственного приложения обусловлена необходимостью локального хранения медиаресурсов на собственных мощностях предприятия, гибкого управления базой пользователей и исключения рисков несанкционированного доступа к служебным или образовательным аудиоматериалам организации.

1.2. Обоснование выбора архитектуры и технологического стека

Для реализации проекта была выбрана классическая трехзвенная клиент-серверная архитектура, включающая в себя уровень представления (мобильный клиент), уровень бизнес-логики (REST API сервер) и уровень хранения данных (СУБД).

В качестве серверного фреймворка выбран FastAPI. FastAPI является одним из наиболее производительных веб-фреймворков на Python благодаря интеграции библиотек Starlette и Pydantic. Использование стандарта асинхронного программирования ASGI (uvicorn) позволяет серверу эффективно обрабатывать тысячи параллельных запросов ввода-вывода (I/O-bound), что критически важно при раздаче аудиофайлов. Для взаимодействия с СУБД используется SQLAlchemy ORM, автоматизирующая построение безопасных SQL-запросов.

Для разработки мобильного интерфейса выбран кроссплатформенный фреймворк Flutter. Использование компилируемого языка Dart обеспечивает высокую производительность графики за счет собственного движка отрисовки Impeller. Flutter позволяет использовать единую кодовую базу для сборки под платформы Android и iOS. Управление состоянием (State Management) реализовано через пакет Provider, разделяющий визуальную верстку и логику работы с данными.

Для хранения структурированной информации выбрана реляционная СУБД MySQL. Она обеспечивает высокую скорость чтения транзакций, надежность хранения данных и полную совместимость со стандартом SQL.

1.3. Спецификация и структура реляционной базы данных

Схема базы данных спроектирована с учетом обеспечения целостности и минимизации избыточности данных. Ниже представлены таблицы 1.3.1 – 1.3.2 физической структуры базы данных MySQL.

Таблица 1.3.1 – Структура таблицы users (учетные записи пользователей)

Содержание поля	Имя поля	Тип, длина	Примечания
Идентификатор пользователя	id	PRIMARY KEY	первичный, авто-инкремент
Электронная почта пользователя	email	VARCHAR(150)	уникальное, не пустое
Хэш-код пароля	hashed_password	VARCHAR(255)	не пустое
Роль доступа	role	VARCHAR(50)	стандартное значение listener

Таблица 1.3.2 – Структура таблицы podcasts (каталог подкастов)

Содержание поля	Имя поля	Тип, длина	Примечания
Идентификатор подкаста	id	PRIMARY KEY	первичный, авто-инкремент
Название аудиозаписи	title	VARCHAR(255)	не пустое
Категория подкаста	category	VARCHAR(100)	не пустое
Описание содержания подкаста	description	VARCHAR(1000)	не пустое
Путь к аудиозаписи на сервере	file_path	VARCHAR(500)	не пустое
Дата и время публикации	created_at	DATETIME	стандартное, текущее время с датой